

实验 2 进程的控制

2.1 实验目的

- (1) 熟练掌握 Linux 环境下全屏幕文本编辑器 vi/vim 的使用;
- (2) 掌握 Linux 系统中的编译器 gcc 的使用方法;
- (3) 加深对进程概念的理解, 明确进程和程序的区别;
- (4) 理解 Linux 下进程创建的原理和特点;
- (5) 掌握 fork()系统调用的使用方法。

2.2 预备知识

2.2.1 vi/vim 编辑器必知必会

1.为什么要学习 vim 编辑器

Linux 的命令行界面下面有非常多的文本编辑器。比如经常听说的就有 Emacs、pico、nano、joe 与 vim 等。vim 可以看做是 vi 的高级版。我们为什么一定要学习 vim 呢? 有以下几个原因:

- (1)所有的 Unix like 系统都会内置 vi 文本编辑器, 其他的文本编辑器则不一定会存在。
- (2)很多软件的编辑接口都会主动调用 vi。
- (3)vim 具有程序编辑的能力, 可以主动以字体颜色辨别语法的正确性, 方便程序设计。
- (4)程序简单, 编辑速度快。

2. 基本使用方法及其相关命令

vim 编辑器的三种模式: 命令模式、编辑模式和末行模式。

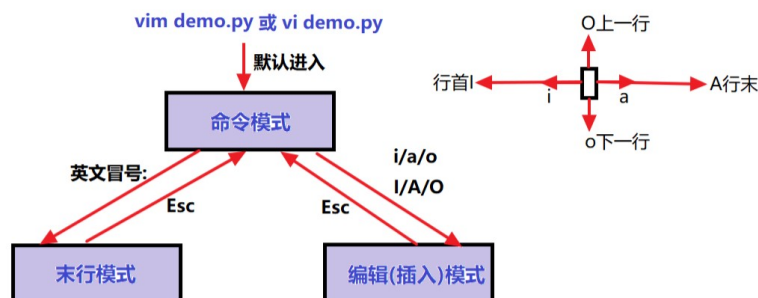
在命令模式中可以进行删除、复制和粘贴的功能, 但是无法编辑文件内容。

从命令模式切换到编辑模式可以按下 i、I、o、O、a、A、r、R 键。按下 Esc 键可以回到一般模式。

在命令模式中输入:、/、? 三个中的任意一个可以将光标移到最下面的一行, 即来到末行模式。在这个模式中可以提供查找数据的操作, 而读取、保存、大量替换字符、离开 vim、显示行号等操作则是在此模式中完成的。

需要注意的是, 编辑模式与末行模式之间是不能互相切换的。

vi/vim有三种基本工作模式



下面列出平时用的最多的命令：

(1)移动光标的方法

[Ctrl]+[f]：屏幕向下移动一页，相当于[PageDown]按键。

[Ctrl]+[b]：屏幕向上移动一页，相当于[PageUp]按键。

0 或功能键[Home]：移动到这一行的最前面字符处。

\$或功能键[End]：移动到这一行的最后面字符处。

G：移动到这个文件的最后一行。

gg：移动到这个文件的第一行，相当于 1G。

N[Enter]：N 为数字，光标向下移动 N 行。

(2)查找和替换

/word：向下寻找一个名称为 word 的字符串。

?word：向上寻找一个名称为 word 的字符串。

:n1,n2s/word1/word2/g：在第 n1 行和 n2 行之间寻找 word1 这个字符串，并且将其替换为 word2。

:1,\$s/word1/word2/g：从第一行到最后一行寻找 word1 这个字符串，并且将其替换为 word2。

:1,\$s/word1/word2/gc：从第一行到最后一行寻找 word1 这个字符串，并且将其替换为 word2。且在替换前显示提示字符给用户确认是否需要替换。

(3)删除、复制和粘贴

x,X：在一行字中，x 为向后删除一个字符（相当于[Del]键），X 为向前删除一个字符（相当于[Backspace]）。

dd：删除光标所在的一整行。

ndd：删除光标所在的向下 n 行。

yy：复制光标所在的一行。

nyy：复制光标所在的向下 n 行。

p,P：p 为将已复制的内容在光标的下一行粘贴，P 则为粘贴在光标的上一行。

u：复原前一个操作。

[Ctrl]+r：重做上一个操作。

.：小数点，重复前一个操作。

(4)一般模式切换到编辑模式

i,I：进入插入模式，i 为从目前光标所在处插入。I 为在目前所在行的第一个非空格字符处开始插入。

a, A：进入插入模式。a 为从目前光标所在处的下一个字符处开始插入。A 为从所在行的最后一个字符处开始插入。

o, O：进入插入模式。o 为在下一行插入。O 为在上一行插入。

r, R：进入替换模式。r 只替换光标所在那个字符一次。R 会一直替换光标所在字符，直到按下 Esc 键。

(5)一般模式切换到命令行

:w: 将编辑的数据写入到硬盘中。

:q: 离开 vi.后面加! 为强制离开。

:wq: 保存后离开。:wq!为强制保存后离开。

目前主要的编辑器都有恢复功能，vim 也不例外。vim 是通过“保存”文件来挽回数据的。

每当我们在用 vim 编辑时，vim 都会自动在被编辑的文件的目录下面再新建一个名为 filename.swap 的文件。这就是一个暂存文件，我们对文件 filename 所做的操作都会被记录到这个文件当中。如果系统意外崩溃，导致文件没有正常保存，那么这个暂存文件就会发挥作用。下面用一个例子来说明。

打开终端，输入命令，将 etc 目录下面的 manpath.config 复制到 tmp 目录下面，并且更改当前工作目录为 tmp:

```
cp /etc/manpath.config /tmp
cd /tmp
```

用 vim 编辑 manpath.config 文件: vim manpath.config。

在 vim 的一般模式下按下 Ctrl+z 组合键，vim 就会被丢到后台执行。回到命令提示符环境后，模拟将 vim 的工作不正常中断。

kill -9 %1;强制杀死制定的进程。

这样导致暂存盘无法通过正常的流程结束，所以暂存文件不会消失，而是继续保留下来。当再次编辑那个文件时(输入命令 vim manpath.config)，出现(ubuntu 11.10):

这时，有六个按钮可以使用:

O(Open for Read-Only):打开成只读文件。

E(edit):用正常方式打开要编辑的文件，并不会载入暂存文件的内容。这很容易出现两个用户相互改变对方的文件的问题。

R(ecover): 加载暂存文件的内容。

D(elete): 如果你确定这个暂存文件是没有用的，则可以删除。

Q(uit): 不进行任何操作，回到命令行。

A(bort): 忽略这个编辑行为，和 Q 类似。

需要注意的是：这个暂存文件不会应为你结束 vim 后自动删除，必须要手动删除。否则每次打开对应的文件时都会出现这样的提示。

3. vim 的功能

(1)块选择

这个功能可以让我们复制一个矩形区域的内容，十分方便。

v:字符选择，会将光标经过的地方反白选择；

V:行选择;

Ctrl+v: 块选择;

y: 复制反白的地方;

d: 删除反白的地方。

需要注意的是, 粘贴时候也是粘贴在一个块的范围, 而不是以行为单位来处理的。

(2)多文件编辑

在两个或多个文件之间复制粘贴内容时, 这个功能会让我们方便很多。

使用命令 `vim name1 name2 name3...`(各个文件名之间用空格隔开)可以同时打开多个文件。

:n: 编辑下一个文件;

:N: 编辑上一个文件;

:files: 列出目前 vim 打开的所有文件。

(3)多窗口功能

可以在一个窗口中打开多个文件。

输入命令 `sp{filename}` 便可以实现这个功能。如果想要在新窗口启动另外一个文件, 则加入文件名。如果省略文件名, 则打开的是同一个文件。

Ctrl+w+j: 先按下 Ctrl 不放, 再按下 w 后放开所有的按键, 再按下 j(或向下箭头键), 则光标可以移到下方的窗口;

Ctrl+w+k: 同上, 不过光标移到上面的窗口;

Ctrl+w+q: 离开。

(4)vim 环境设置

需要注意的是, vim 会将我们的以前的行为都记录下来, 以方便我们操作。它保存在文件: `~/.viminfo` 中。

vim 常用的环境设置参数命令如下:

:set nu 设置行号

:set nonu 取消行号

:set hlsearch 设置高亮度查找

:set nohlsearch 取消高亮度查找

:set backup 自动备份文件

:set ruler 开启右下角状态栏说明

:set showmode 显示左下角的 INSERT 之类的状态栏

:set backspace={0,1,2} 设置退格键功能。为 2 时可以删任意字符。为 0 或 1 时仅可以删除刚才输入的字符。

:set all 显示目前所有的环境参数值

:set 显示与系统默认值不同的参数值

:syntax on/off 是否开启依据相关程序语法显示不同的颜色

:set bg=dark/light 是否显示不同的颜色色调

但是我们没有必要每次使用 vim 都要重新设置一次各个参数值。我们可以通过配置文件来直接规定我们习惯的 vim 操作环境。整体 vim 的设置值一般是放在 `/etc/vimrc` 中的。我们一般不要修改这个文件。我们可以通过修改 `~/.vimrc` 这个文件, 如果不存在, 可以手动创建。然后将我们所希望的设置值写入。例如, 可以这样写:

```
vim ~/.vimrc
set hlsearch（注意：set 前面也可以加冒号，结果一样）
set backspace=2
set ruler
set showmode
set nu
syntax on
```

创建并保存这个文件之后，当下次重新以 vim 编辑某个文件时，该文件的默认环境就是这么设置的。

2.2.2 进程的创建

Linux 操作系统是面向多用户的。在同一时间可以有許多用户向操作系统发出各种命令。那么操作系统是怎么实现多用户的环境呢？在现代的操作系统里面，都有程序和进程的概念。那么什么是程序，什么是进程呢？通俗的讲程序是一个包含可以执行代码的文件，是一个静态的文件。而进程是一个开始执行但是还没有结束的程序的实例，就是可执行文件的具体实现。

一个程序可能有许多进程，而每一个进程又可以有许多子进程。依次循环下去，产生子孙进程，形成进程家族。

当程序被系统调用到内存以后，系统会给程序分配一定的资源（如内存、设备等等），然后进行一系列的复杂操作，使程序变成进程以供系统调用。在系统里面只有进程没有程序，为了区分各个不同的进程，系统给每一个进程分配了一个 ID（就象我们的身份证）以便识别，就是 PID。

为了充分的利用资源，系统还对进程的状态进行不同区分，将进程分为新建，运行，阻塞，就绪和完成五个状态，以及两个挂起状态。新建表示进程正在被创建，运行是进程正在运行，阻塞是进程正在等待某一个事件发生，就绪是表示系统正在等待 CPU 来执行命令，而完成表示进程已经结束了系统正在回收资源。关于进程状态的详细解说可以看《计算机操作系统》上面详细的说明。

创建一个进程的系统调用很简单，只要调用 fork 函数就可以了。

```
pid_t fork();
```

当一个进程调用了 fork 以后，系统会创建一个子进程，子进程和父进程只是进程 ID 不同，其他的都是一样，就像是进程克隆（clone）了自己一样。为了区分父进程和子进程，我们必须跟踪 fork 的返回值。这是 fork 函数很独特的地方，希望大家好好体会。

当 fork 调用失败（内存不足或者是用户的最大进程数已到），fork 返回-1；否则 fork 的返回值有重要的作用。对于父进程 fork 返回子进程的 ID（大于 0），而对于子进程 fork 返回 0。我们就是根据这个返回值来区分当前是处于父进程还是子进程。

父进程为什么要创建子进程呢？前面我们已经说过了 Linux 是一个多用户操作系统，在同一时间会有许多的用户在争夺系统的资源，有时进程为了早一点完成任务就创建子进程来争夺资源。一旦子进程被创建，父子进程一起从 fork 处继续执行，相互竞争系统的资源。

2.2.3 Linux 下的编译器 gcc

C 语言是 Linux 下的最常用的程序设计语言，Linux 上的很多应用程序都是用 C 语言编写的。Linux 系统上运行的 GNC C 编译器（gcc）是一个全功能的 ANSI C 兼容编译器。虽然 gcc 没有集成的开发环境，但堪称目前效率很高的 C/C++编译器。

命令格式: gcc [选项] 源文件

选项含义:

-o FILE 指定输出文件名, 在编译为目标代码时, 这一选项不是必须的。如果 FILE 没有指定, 缺省文件名是 a.out.

-c 只编译不链接

-O 优化编译过的代码

-w 关闭所有警告, 建议不要使用此项

2.3 实验内容

1.了解系统调用 fork()、exec()、exit()和 wait()的功能和实现过程。

2.编写一段程序, 使用系统调用 fork()来创建两个子进程, 并由父进程重复显示字符串“parent:”和自己的标识数, 而子进程则重复显示字符串“child:”和自己的标识数。

3.编写一段程序, 使用系统调用 fork()来创建一个子进程。子进程通过系统调用 exec()更换自己的执行代码, 新的代码显示“new program.”。而父进程则调用 wait()等待子进程结束, 并在子进程结束后显示子进程的标识符, 然后正常结束。

2.4 实验指导

1.假设要写一个 hello.c 文件, 首先使用 vi hello.c 命令, 然后用 I 命令或者按 insert 进行编辑, 内容如下:

```
#include<stdio.h>
int main(){
    printf("Hello Linux!\n");
    return 0;
}
```

2.按 ESC, 在 vi 命令模式下使用: wq 命令进行保存, 并退出编辑器;

3.进入 hello.c 所在目录, 使用 gcc -o hello hello.c 对源文件进行编译。该命令的意思就是对 hello.c 文件进行编译, 声明生成的目标文件名为 hello (参考 -o 选项的说明)。

4.要看执行结果, 使用 ./hello 就可以在屏幕上看到输出结果了。

5.关于 fork()函数

fork 函数用于创建一个新的进程 (子进程), 其调用格式为:

```
pid_t fork();
```

正确返回:

等于 0, 创建了子进程, 并且是在子进程中返回, 即接下来执行子进程的代码

大于 0, 创建了子进程, 从父进程中返回子进程的 ID 值。

错误返回:

等于-1, 创建失败。

子进程被创建后, 就进入就绪队列, 和父进程一起独立地等待调度。子进程继承父程序的程序段代码, 子进程被调度执行后, 也会和父进程一样, 从 fork()返回。**从共享程序段代码的角度看**, 父子进程执行的程序段代码是同一个, 在内存中只有一个程序段副本; 但是**从编程的角度来看**, 为了使父子进程作不同的事情, 要在程序中区分父进程和子进程的代码段, 需要借助于从 fork()返回的值, 来标志当前进程的身份。

从 fork() 返回后，都会执行如下语句： pid = fork();

得到返回的值 pid 后，有三种情况：

1) 若 pid < 0，则表示 fork() 出错，执行

```
if( pid<0){  
    printf("fork error\n");  
    exit(0);  
}
```

2) 若 pid == 0，表示当前进程是子进程，后面执行的代码，是子进程要做的事情

```
if( pid == 0){  
    printf("Message from Child!");  
    exit(0);  
}
```

3) 若 pid > 0，表示当前进程是父进程，后面执行的代码，是父进程要做的事情

```
if( pid > 0){  
    printf("Message from Parent!");  
    exit(0);  
}
```

由于父子进程分别独立地进入就绪队列等待调度，所以谁会先得到调度，是不确定的，这与系统的调度策略和系统当前资源状态有关，因此谁先从 fork() 返回，继续执行后面的语句，也是不确定的。

2.5 参考源代码

//代码 1

```
1 #include<stdio.h>  
2 #include<unistd.h>  
3  
4 int main()  
5 {  
6     pid_t pid1,pid2;  
7     pid1=fork();  
8     if(pid1<0)  
9     {  
10         printf("fork error");  
11     }  
12     else if(pid1==0)  
13     {  
14         printf("child pid1 : %d\n",getpid());  
15         printf("pid1 of myparent id is: %d\n",getppid());  
16         sleep(3);  
17     }  
18     else  
19     {  
20         printf("parent process is running\n");  
21         pid2=fork();  
22         if(pid2<0)  
23         {  
24             printf("fork error\n");  
25         }  
26     }  
27 }
```

```

26     else if(pid2==0)
27     {
28         printf("child pid2 : %d\n",getpid());
29         sleep(3);
30     }
31     else
32     {
33         printf("parent id : %d\n",getpid());
34         sleep(3);
35     }
36 }
37 return 0;
38 }

```

//代码 2

```

1  /*代码2主程序
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<unistd.h>
5  #include<sys/types.h>
6  #include<sys/wait.h>
7
8  int main()
9  {
10     pid_t pid;
11     pid=fork();
12     if(pid<0)
13     {
14         printf("fork error\n");
15     }
16     else if(pid==0)
17     {
18         // if(execlp("/home/lgx/goto","",NULL)<0)
19         // {
20         //     printf("execlp error\n");
21         //     exit(1);
22         // }
23         //execl("/home/lgx/goto","",NULL);
24         execl("/bin/ls","ls","-l",NULL);
25         exit(0);
26     }
27     else
28     {
29         printf("child pid : %d\n",waitpid(pid,NULL,WUNTRACED));
30         exit(0);
31     }
32     return 0;
33 }

```

```

1  //被代码2调用的goto.c程序
2  #include<stdio.h>
3  int main()
4  {
5     printf("i am is newcomer.\n");
6     return 0;
7 }

```